

ECDAT — The Complete Team Guide

Smart India Hackathon 2026 · Problem Statement SIH26164 · National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity · Category: Software · Team Asterisks

Last updated 4 September 2026. Everything in this guide was checked against the actual code and

data in this repository. Where something is planned rather than built, it says so.

How to use this document

Read **Part 1** and **Part 2** before your first team meeting — that is about 25 minutes and it will let you follow any conversation about this project. Read **Part 3** when you want to know what we have actually built. Read **Part 8** when it is your turn to build something.

You are not expected to already know cryptography. This guide assumes you know nothing about it and

explains everything from the beginning. Nothing here is written to impress you; it is written so you can repeat it to a stranger.

Contents

Part	What it covers
1	ECDAT in plain English — the five-minute version
2	The vocabulary — every technical word we use, explained simply
3	The current state — what exists today, screen by screen and file by file
4	The workflow — how a folder of code becomes a ranked list of risks
5	The concepts and the maths, worked through with real numbers
6	The technical stack — what we use now, what we would use at scale
7	How to run it on your own machine
8	How to build and extend it — the implementation guide
9	Future plans — the roadmap after the hackathon
10	Splitting the work across six people
11	The honesty rules — claims we make and claims we refuse to make

Part	What it covers
12	Quick reference card

Part 1 — ECDAT in plain English

1.1 The problem, told as a story

Imagine a bank. Every day it moves money, checks identities, and stores customer records. All of that is protected by **cryptography** — mathematical locks that scramble data so only the right person can read it.

The bank has been running for twenty years. In that time, hundreds of developers have written millions of lines of code. Each one of them, at some point, picked a lock and used it. Some picked good locks. Some picked locks that were fine in 2008 and are embarrassing now. Nobody wrote down which lock went where.

Now ask the bank's Chief Information Security Officer a simple question:

"Which cryptographic algorithms are running in your organisation right now, and what does each one protect?"

They cannot answer. Not because they are careless, but because **nobody has an inventory**. The information exists, but it is scattered across source code, configuration files, certificates and libraries, and no human can read all of it.

That is already a problem. Here is what makes it urgent.

1.2 The quantum deadline

A **quantum computer** is a fundamentally different kind of computer. Large ones do not exist yet.

When one does exist at sufficient scale — the industry calls it a **CRQC**, a *Cryptographically Relevant Quantum Computer* — it will break a specific family of locks completely. Not weaken them: break them.

The family it breaks is **public-key cryptography**: RSA, ECDSA, ECDH, Diffie–Hellman. These are the locks that protect essentially every HTTPS connection, every digital signature, and every secure key exchange on the internet today.

Nobody knows when the CRQC arrives. Estimates run from ten to thirty years. And here is the cruel part, called **"Harvest Now, Decrypt Later"** (HNDL):

An adversary does not need to wait. They can record your encrypted traffic **today**, store it, and decrypt it in fifteen years when the machine exists.

So if you are protecting something that must stay secret for twenty years — medical records, identity data, sealed financial archives, defence documents — **you are already exposed**, right now, even though the quantum computer does not exist yet.

The world's answer is **Post-Quantum Cryptography (PQC)**: new locks, designed to resist quantum computers, that run on ordinary hardware. The US standards body NIST finalised the first three in 2024. Migrating to them is now a multi-year programme for every serious organisation on earth.

But you cannot migrate what you cannot find. Discovery comes first. That is the problem statement NTRO issued, and that is ECDAT.

1.3 What ECDAT actually does

ECDAT is a tool you point at a codebase. It does four things, in order:

Step	Question it answers	What it produces
1. Discover	Where is cryptography used in this code?	A finding with file, line, column, the exact source text, and the enclosing function
2. Classify	What does that cryptography protect, and how much does it matter?	A business-criticality score, with the confidence of that score labelled honestly
3. Prioritise	Which of these must be fixed first, and is anything already too late?	A risk score and a Mosca verdict per service
4. Measure	What would replacing it actually cost?	A Migration Passport with real measured performance and size numbers

1.4 Why this is not just another scanner

This is the single most important thing every one of us must be able to say. Tools that do **step 1** already exist and some are free — IBM Research's CBOMkit is the obvious one. If we claim step 1 as our innovation, a knowledgeable judge will dismiss us in one question.

Our contribution is **steps 2, 3 and 4**:

- **Step 2** — Every commercial scanner stops at *technical* severity ("this is RSA-2048, that is medium risk"). None of them answer "*what does this protect?*" ECDAT reads a **service manifest** written by the asset owner, so a finding in the KYC service is scored differently from an identical finding in a test fixture. And when there is no manifest, it says `unknown` rather than inventing a number.
- **Step 3** — ECDAT implements **Mosca's inequality** as a screen you can touch, not logic buried in a config file. The problem statement names this framework explicitly.
- **Step 4** — ECDAT *measures* the cost of migrating rather than quoting a vendor datasheet. On our machine it found that switching to post-quantum signatures makes signing 1.3× slower (irrelevant) but makes the signature **12.9× larger** (a serious problem). Speed is not the blocker. Size is. That finding only exists because the tool measured instead of guessing.

1.5 The one-sentence pitch

ECDAT is a local-first tool that finds cryptography in source code, works out what it protects, decides how urgently it must be replaced, and measures what replacing it would cost — with every number traceable back to a line of real code.

Part 2 — The vocabulary

Every word below appears somewhere in our slides, our code, or the questions a judge will ask. Learn these and you can hold any conversation about this project.

2.1 Cryptography basics

Cryptography — using mathematics to protect information. Three jobs: keep it secret (*encryption*), prove it was not altered (*integrity*), prove who sent it (*authentication*).

Symmetric encryption — one shared key locks and unlocks. Fast. Example: **AES**. Like a padlock

where both people have a copy of the same key. The problem: how do you give the other person the key

safely in the first place?

Asymmetric / public-key cryptography — two mathematically linked keys. A **public key** anyone

can see, and a **private key** only you hold. Anything locked with one is unlocked by the other.

This solves the key-sharing problem and is what makes the internet work. Examples: **RSA**, **ECDSA**, **ECDH**, **Diffie–Hellman (DH)**.

Hashing — a one-way fingerprint of data. The same input always gives the same fingerprint; you

cannot go backwards. Used to check integrity and store passwords. Examples: **MD5** (broken), **SHA-1** (broken), **SHA-256** (fine).

Digital signature — you hash a document, then encrypt the hash with your private key. Anyone with your public key can verify it was you and that nothing changed. Used for software updates, certificates, and legal documents.

Key exchange / key establishment — the handshake two computers do to agree on a shared secret

over an insecure network. **ECDH** and **DH** do this. **ML-KEM** is the post-quantum replacement.

Certificate (X.509) — a file that says "this public key belongs to this website," signed by a trusted authority. What your browser checks when it shows a padlock. Governed by **RFC 5280**.

TLS — the protocol behind HTTPS. It uses key exchange to agree a secret, certificates to prove

identity, and symmetric encryption to protect the actual data.

2.2 The quantum part

Shor's algorithm (1994) — a quantum algorithm that **completely breaks** RSA, ECDSA, ECDH, DH

and Ed25519. It solves the specific hard maths problem those systems rely on. This is the whole

threat.

Grover's algorithm (1996) — a quantum algorithm that **halves the effective strength** of symmetric ciphers and hashes. AES-256 behaves like AES-128 against it. That is a weakening, not a break — AES-128 is still strong. This distinction matters enormously and most people get it wrong.

CRQC — *Cryptographically Relevant Quantum Computer*. One big and stable enough to actually run Shor's algorithm against real key sizes. Does not exist yet.

HNDL — Harvest Now, Decrypt Later — recording encrypted data today to decrypt it after the CRQC arrives. This is why long-lived secrets are urgent *now*.

PQC — Post-Quantum Cryptography — new algorithms designed to resist quantum attack, running on normal computers. Note: PQC is **not** quantum cryptography and does **not** require a quantum computer to use.

Crypto-agility — building systems so the algorithm can be swapped without rewriting the application. The long-term goal; the reason inventory matters.

2.3 The NIST standards

In August 2024 NIST published the first finalised PQC standards. These are the replacements:

Standard	Algorithm	Replaces	What it does
FIPS 203	ML-KEM (was "Kyber")	ECDH, DH, RSA key transport	Key establishment
FIPS 204	ML-DSA (was "Dilithium")	RSA signatures, ECDSA, Ed25519	Digital signatures
FIPS 205	SLH-DSA (was "SPHINCS+")	—	Backup signature scheme, hash-based, very conservative

NIST IR 8547 — the transition timeline. Deprecate classical public-key crypto by **2030**, disallow it by **2035**. This is what our Mosca screen anchors its default assumption to.

NSA CNSA 2.0 — the US National Security Agency's migration schedule, targeting **2033** for national-security systems. This is the firmest published deadline that exists anywhere. Note carefully: it binds *US national-security systems*, not Indian enterprises. We must never misdescribe this.

NIST SP 800-57 — key-management guidance including minimum key sizes. Our certificate checks use it.

Hybrid mode — using a classical and a post-quantum algorithm together, so you are safe if either survives. **X25519MLKEM768** is the pairing Chrome and Cloudflare already run in production. It is a real, deployed thing, not a marketing idea.

2.4 The inventory part

CBOM — Cryptographic Bill of Materials — a structured, machine-readable list of every cryptographic asset in a system: which algorithms, where, in what role. Think of it as an ingredients label for cryptography.

CycloneDX 1.6 — the open standard (from OWASP) that defines the CBOM format. Version 1.6 added a

`cryptographic-asset` component type in 2024 specifically for this job. We emit this standard rather

than our own JSON shape, because *a catalogue nobody else can read is not a catalogue*.

SBOM — the older, more general "Software Bill of Materials" listing software components. CBOM is its cryptographic sibling.

2.5 The scanning part

Static analysis — reading source code without running it. Fast, safe, and works on code you cannot execute. Its ceiling: it cannot see anything decided at runtime.

AST — Abstract Syntax Tree — the structured tree a parser builds from source code. Instead of

treating `rsa.generate_private_key(key_size=2048)` as a string, an AST lets you ask "*what was the*

key_size argument?" and get the answer `2048` reliably. Python has this built in (the `ast` module). This is what separates us from a keyword grep.

Regex (regular expression) — a pattern for finding text. Cheap and fast, but it does not understand code structure. We use regex for *triage* (finding candidates) and AST for *resolution* (extracting the actual values).

Triage — cheaply eliminating files that cannot possibly contain what you are looking for, before doing expensive work. Our scanner throws away files that contain none of a short list of literal

substrings any crypto call must contain.

Incremental scan — hashing each file's contents and skipping any file whose hash has not changed

since the last run. Measured at **8× faster** than a clean scan on our corpus.

False positive — the tool reports something that is not really there. **False negative** — the tool misses something that is there. Both matter; a security tool with many false positives gets ignored.

Confidence label — our way of being honest. Every criticality score carries one of:

`manifest-confirmed` (an owner declared it), `path-heuristic` (a keyword matched, and we show you which keyword), or `unknown` (neither — we say so rather than guessing).

2.6 The prioritisation part

Business criticality — how much a system matters to the organisation. Not a technical property.

A weak algorithm in a test fixture and the same algorithm in a payments service are the same *finding* but completely different *problems*.

Mosca's inequality — proposed by Michele Mosca (2018, *IEEE Security & Privacy*). Three numbers:

- **X** = how long your data must stay secret (or your signature stay unforgeable)
- **Y** = how long your migration will realistically take
- **Z** = how long until a CRQC exists

If $X + Y > Z$, you are already too late.

Read it aloud: *"if the time my data must stay secret, plus the time it takes me to fix it, is longer than the time I have — I should already have started."*

Threat horizon (Z) — an assumption, not a prediction. We make it a slider precisely because nobody knows. Any tool that presents Z as a confident number should not be trusted.

Air-gapped — a machine with no network connection at all. Our tool runs fully with the network cable out, which is the deployment posture a national security organisation actually needs.

Part 3 — The current state: what exists today

This section is the honest inventory. If you are asked "what have you actually built?", this is the answer.

3.1 The headline numbers (all measured, all real)

Metric	Value
Repositories scanned	4
Source files scanned	3,111 (out of 7,589 files walked)
Bytes scanned	20.7 MB
Findings produced	206
Quantum-vulnerable findings	120
X.509 certificates parsed	17
Dependencies identified	16
Detection rules	56
Algorithms in the knowledge base	37
Clean scan time	3.77 s
Incremental rescan time	0.47 s (8× faster)
Benchmark time	8.69 s
Total pipeline	~13 s

3.2 The four scan targets

Target	What it is	Findings	Why it is in the demo
nivesh-core	Our own seven-service financial demo estate, with a completed service manifest	71	The only target where the full criticality + Mosca chain can be shown end to end
paramiko	Real upstream open-source SSH library (Python), cloned unmodified	55	Dense in key-exchange crypto. No manifest — proves honest degradation
jjwt	Real upstream Java JWT library, cloned unmodified	49	Signature-algorithm selection is its whole job
django	Real upstream Python web framework, cloned unmodified	31	Large, well-known codebase; contains genuine MD5/SHA-1 legacy paths

Critical honesty point: *Nivesh Financial Services is fictional.* Its code, certificates and configuration are real files that are really scanned — but the organisation and the manifest describing it were written by us for this demonstration. The other three are genuine upstream repositories that we did not touch. We say this out loud, every time.

3.3 The seven screens

The dashboard has seven screens. Each answers one question.

1 · **Command Centre** — *"What is the overall posture?"*

Headline counts, the quantum status of every finding as a stacked bar, risk distribution as a donut, algorithms in use, the highest-priority findings, and scan telemetry measured on the machine.

2 · **Crypto Inventory** — *"Show me everything, and prove each one."*

A filterable, sortable table of all findings. Click any row and an **evidence drawer** opens showing the file, line, column, the matched source text with the match highlighted, the enclosing function, the rule ID that fired, and the complete arithmetic behind both the criticality score and the risk score. Nothing is hidden.

3 · **Business Criticality** — *"What does this cryptography protect?"*

The scoring weights, fully visible. The seven services with their individual scores and the arithmetic shown. A heatmap of which vulnerable algorithm lives in which service. On a target with no manifest, this screen instead explains the fallback and shows exactly which keywords matched.

4 · **Mosca Horizon** — *"Who is already late?"*

The inequality explained, a Z slider (3–35 years) with preset anchors, and a live per-service verdict with a visual timeline. Every X and Y is also draggable.

5 · **Migration Passport** — *"What does fixing it cost?"*

Side-by-side measured comparisons for signing and key establishment, full measurement tables, certificate size bars, the TCP congestion-window consequence, and a downloadable machine-readable passport record.

6 · **CBOM Export** — *"Can other tools read your output?"*

The CycloneDX 1.6 document, component breakdown, and a download button.

7 · Method & Limits — "What does this NOT do?"

How a finding is produced, what is real versus staged, and six named limitations.

Plus a **Guided walkthrough** — a presenter mode that drives all of the above from the arrow keys.

See Part 3.5.

3.4 The code, file by file

```
proto/
├─ engine/                                the Python analysis pipeline
│   └─ rules.py          (27 KB)  KNOWLEDGE BASE. 37 algorithm profiles + 56
detection rules.
│   │
│   │   Records for each primitive how Shor/Grover
affects it, and
│   │
│   │   what the NIST replacement is. Also decodes
Java JCA strings
│   │
│   │   ("SHA256withRSA") and TLS cipher suite names.
│   └─ scanner.py      (17 KB)  THE SCANNER. Walks files, runs the four-stage
detection
│   │
│   │   pipeline, produces evidence-carrying findings.
│   └─ criticality.py (8 KB)  THE CRITICALITY ENGINE. Loads the service
manifest, matches
│   │
│   │   file paths to services, computes the
criticality score, and
│   │
│   │   evaluates Mosca's inequality.
│   └─ artefacts.py    (9 KB)  X.509 certificate parsing (key size, Key
Usage, expiry) and
│   │
│   │   dependency-manifest reading (requirements.txt,
pom.xml).
│   └─ cbom.py         (8 KB)  CycloneDX 1.6 CBOM export.
│   └─ bench.py        (13 KB)  THE BENCHMARK. Really measures
RSA/ECDSA/Ed25519/ECDH vs
│   │
│   │   ML-Dsa/ML-KEM, and issues real X.509
certificates under each
│   │
│   │   algorithm to weigh them.
│   └─ run.py          (14 KB)  THE ORCHESTRATOR. Runs everything, computes
risk scores,
│   │
│   │   writes ui/data.js + data/scan_results.json +
the CBOMs.
│   └─ build_artifact.py
│
├─ ui/
step
└─ index.html                the shell (open this)
```

```

|   |─ app.js          (82 KB)  all seven views, the evidence drawer, the
guided walkthrough
|   |─ styles.css      (28 KB)  the design system
|   |─ data.js         (455 KB) GENERATED by run.py – never edit by hand
|   └─ ecdat-console.artifact.html    single-file build for sharing
|
|─ targets/              what gets scanned
|   |─ nivesh-core/      our demo estate + ecdat-manifest.yaml
|   └─ paramiko/ jjwt/  django/    real upstream repos, cloned unmodified
|
|─ data/                 generated outputs
|   |─ scan_results.json  the full machine-readable scan record
|   └─ cbom-<target>.json one CycloneDX CBOM per target
|
└─ README.md             run instructions + the jury script

```

3.5 The guided walkthrough (our strongest demo asset)

The console drives its own demonstration. Press ► **Guided walkthrough** in the top bar and a presenter rail appears at the bottom of the screen with seven steps. Each step **automatically sets the target, the view, and any filter it needs** — so nobody has to hunt for a control in front of a jury.

Key	Action
→ or Space	next step
←	previous step
1 – 7	jump straight to a step
Esc	close the evidence drawer, then leave the walkthrough

The seven steps:

1. **Command Centre**, all targets — *"we scanned real code"*
2. **Crypto Inventory**, Nivesh, critical filter — *the drawer opens automatically on the SHA-1 over-Aadhaar finding*
3. **Business Criticality**, Nivesh — *"what does this protect?"*
4. **Business Criticality**, paramiko — *"and what happens when nobody declared it"*
5. **Mosca Horizon**, Nivesh — *"who is already late"*
6. **Migration Passport** — *"measured, not estimated"*
7. **Method & Limits** — *"what this does not do"*

This means the demo cannot get lost, and any team member can drive it after ten minutes of practice.

It is also, itself, a differentiator worth mentioning: the tool ships knowing how to explain itself.

3.6 What is real, and what is staged

Real:

- 3,111 source files from paramiko, JJWT and Django, cloned unmodified from upstream
- 206 findings, each with file, line, column and the source text that matched
- 17 X.509 certificates parsed from disk — real key sizes, real Key Usage flags, real expiry
- Every benchmark number, measured with OpenSSL's native ML-KEM and ML-DSA during the run
- Scan timings including the 8× incremental speedup, measured back to back

Staged for the demonstration:

- Nivesh Financial Services is fictional (its files are real and really scanned; the organisation and manifest are ours)
- The service manifest is filled in by hand — CMDB integration does not exist yet
- No live TLS handshake — the benchmark measures algorithms directly

Does not exist at all (and we must never imply otherwise):

- Code generation / codemod / patch output
- Language support beyond Python, Java, TLS config and X.509
- Parallel or multi-threaded scanning (it is single-threaded; the speedup comes from incremental hashing)
- Sandboxed migration rehearsal against a live endpoint
- Any LLM in the detection or scoring path

Part 4 — The workflow

This is what happens when you run `python engine/run.py`. Follow it once and the whole product makes sense.

4.1 The pipeline, end to end

targets/<repo>/



STEP 1 · ENUMERATE

Walk the directory. Skip .git, node_modules, build, dist, vendor, site-packages, migrations... Classify each file: python / java / config / certificate / dependency-manifest. Skip anything over 1.5 MB.

→ 7,589 files walked, 3,111 kept



STEP 2 · TRIAGE (cheapest possible filter)

Does this file contain ANY of a short list of literal substrings that any crypto call must contain?

("hashlib", "Cipher", "generate_private_key", "padding.", "algorithms.", "hmac", ...)

If not → discard immediately, do no parsing at all.



survivors only



STEP 3 · RULES (56 compiled regex patterns)

Locate candidate call sites. Where a call declares its own algorithm, DECODE it rather than guess:

Signature.getInstance("SHA256withRSA")

→ algorithm RSA, digest SHA-256, purpose signature

ssl_ciphers "ECDHE-RSA-AES128-SHA"

→ ECDH key exchange + RSA auth + AES-128 + SHA-1



files that produced a hit only



STEP 4 · PARAMETER RESOLUTION

Python → parsed with the 'ast' module to recover concrete arguments: key_size=2048, ec.SECP256R1(), and the name of the enclosing function.

Java → bounded look-around (±6 lines) for the matching initialize(2048), plus a symbol index for method names.

The index is built LAZILY – only once a rule has already hit.



STEP 5 · CLASSIFY THE ALGORITHM

Look the primitive up in the 37-entry knowledge base:

shor-broken / classically-broken / grover-reduced /
quantum-safe, plus the NIST replacement.



STEP 6 · CLASSIFY THE BUSINESS CONTEXT

Match the file path against ecdat-manifest.yaml.

match → manifest-confirmed, use the declared values

no match, keyword hits → path-heuristic, show the keyword

neither → unknown, and say so

Compute criticality = (data + function) × exposure



STEP 7 · MOSCA + RISK

X + Y vs Z per service → exposed or clear, and by how much.

risk = quantum_urgency × (criticality/20) × 2 + mosca_press.



STEP 8a · CERTIFICATES & DEPENDENCIES

Parse every .pem/.crt: algorithm, key size,
Key Usage flags (RFC 5280), expiry.

Read requirements.txt / pom.xml for crypto libs.

STEP 8b · BENCHMARK (once per run, not per repo)

Really sign/verify/exchange with RSA-2048/3072,
ECDSA-P256, Ed25519, ML-DSA-44/65/87, ECDH-P256/
P384, X25519, ML-KEM-768/1024.

Warm up 5×, then sample until BOTH 30 iterations
and 0.35 s have elapsed. Report the median.

Then ISSUE a real X.509 certificate under each
algorithm and weigh the DER encoding.

STEP 9 · EMIT

ui/data.js → the dashboard reads this

data/scan_results.json → the full machine-readable record

data/cbom-<repo>.json → one CycloneDX 1.6 CBOM per target

4.2 What one finding actually contains

Every finding is a record like this (this is the real top finding):

```
id                f-2d0504b70feb
repo              nivesh-core
file              services/kyc-service/app/identity_vault.py
line / col        72 / 11
snippet           return hashlib.sha1(aadhaar_number.encode()).hexdigest()
match             hashlib.sha1(
rule_id           py-hash-sha1
symbol            legacy_dedupe_key          ← the enclosing function
language          python
algorithm         SHA-1
function          hashing                    ← the purpose
quantum           classically-broken
criticality        score 18, bucket "critical", confidence "manifest-confirmed"
                  evidence: manifest entry "kyc-service" matched services/kyc-
service/**
                  working: data sensitive-pii = 5, function authentication =
4,
                  exposure internet-facing = x2 → (5+4)×2 = 18
mosca             X=8, Y=2, X+Y=10, Z=15 → clear by 5 years
risk              score 9.0, bucket "critical"
                  working: 5 × 0.9 × 2 = 9.0, mosca pressure 0
```

Notice: **every single number can be recomputed by hand from the values shown.** There is no model, no black box, and nothing an auditor cannot check. That is the design principle.

Part 5 — The concepts and the maths

5.1 The criticality score

This lives in `engine/criticality.py`. Every weight is a plain number you can read and change.

The formula:

```
criticality = (data_classification_weight + business_function_weight) ×
exposure_multiplier
```

The weights:

Data classification	Weight
public	1
internal	2
financial	4
sensitive-pii	5

Business function	Weight
test	0
internal-tooling	1
unknown	2
records-archival	3
payment-processing	4
authentication	4

Exposure	Multiplier
offline	×1.0
internal	×1.5
unknown	×1.5
internet-facing	×2.0

The buckets: below 3 = low · 3–7 = medium · 7–12 = high · above 12 = critical

Worked example — the KYC service:

It handles Aadhaar numbers (`sensitive-pii` = 5), its job is identity checking (`authentication` = 4), and it is reachable from the internet (`internet-facing` = ×2).

```
(5 + 4) × 2 = 18 → "critical"
```

Worked example — the test fixtures:

Fake keys (`public` = 1), test code (`test` = 0), never deployed (`offline` = ×1.0).

```
(1 + 0) × 1.0 = 1 → "low"
```

That is the entire point. The same weak algorithm in both places produces two completely different priorities, because ECDAT knows what each one protects.

5.2 Confidence — the honesty mechanism

The score above is only trustworthy if you know where its inputs came from. Every criticality carries a confidence label:

Label	Meaning	Example evidence shown
manifest-confirmed	The asset owner declared this path in <code>ecdat-manifest.yaml</code>	<i>"manifest entry 'kyc-service' matched services/kyc-service/**"</i>
path-heuristic	No manifest entry, but a keyword in the path matched	<i>"path contains 'tests' → test"</i>
unknown	Neither. ECDAT prints <code>unknown</code> rather than inventing a number	<i>"no manifest entry and no path keyword matched"</i>

On paramiko this produces **0 manifest-confirmed, 17 path-heuristic, 38 unknown** — and that is a feature, not a failure. A tool that produced confident criticality scores for a repository whose owner it has never met would be lying.

5.3 Mosca's inequality

if $X + Y > Z$ then you are already too late

- **X** — data lifetime, in years. How long this must stay confidential or unforgeable.
- **Y** — migration time, in years. How long replacement realistically takes, including counterparties and hardware.
- **Z** — threat horizon, in years. When a CRQC might exist. **An assumption, set by the operator.**

Our demo estate's Z default is **15 years**, and the manifest records *why*: *"Board risk appetite, anchored to NIST IR 8547 (deprecate classical PKC by 2030, disallow by 2035)."* The basis is printed on screen. Z is a slider because honesty demands it.

The seven services, and the moment worth rehearsing:

Service	Criticality	X	Y	X+Y	vs Z=15	Verdict
kyc-service	critical (18)	8	2	10	-5	clear
payment-api	critical (16)	7	3	10	-5	clear
shared-tls-infrastructure	critical (16)	7	3	10	-5	clear
citizen-portal	high (12)	2	1	3	-12	clear
treasury-archive	high (8)	25	4	29	+14	EXPOSED
admin-console	medium (4.5)	1	1	2	-13	clear
test-fixtures	low (1)	0	0	0	-15	clear

treasury-archive scores only "high" on criticality — it is an offline system — yet it is the one

service already 14 years past the line, because its records must stay sealed for 25 years and re-sealing the archive takes 4. Meanwhile payment-api scores "critical" and has 5 years of slack.

Severity and urgency are different questions. Only the second one tells you what to start on

Monday. This single screen is our best argument, because no severity-only scanner can produce it.

5.4 The risk score

```
risk = quantum_urgency × (criticality_score / 20) × 2 + mosca_pressure
```

Quantum urgency (from `run.py`):

Quantum status	Urgency	Why
classically-broken	5	Broken today . No quantum computer required.
shor-broken	4	Broken by a future CRQC.
grover-reduced	2	Weakened, not broken.
quantum-safe	0	Inventory only — not migration work.

Note that `classically-broken` outranks `shor-broken`. MD5 and SHA-1 are exploitable now; a threat

that needs hardware nobody has built is *less* urgent. **A tool that files MD5 under "future**

quantum

risk" has mis-ranked it. This is a deliberate, defensible design decision and a great thing to point at.

Mosca pressure adds up to 3 extra points when a service is past the line, scaled by how far past

(gap_years / 5 , capped at 3).

Buckets: ≥ 8 critical · ≥ 5.5 high · ≥ 3 medium · ≥ 0.8 low · else informational.

Worked example — the SHA-1 Aadhaar finding:

```
quantum_urgency (classically-broken)      = 5
criticality_factor (18 / 20)              = 0.9
base = 5 × 0.9 × 2                        = 9.0
mosca_pressure (kyc-service is clear)     = 0
risk = 9.0 → critical
```

5.5 The Migration Passport measurements

Every number below was measured on the demo machine using OpenSSL 4.0.1's native ML-KEM and ML-DSA.

Signatures:

Algorithm	Sign (ms)	Verify (ms)	Signature (bytes)	Public key (bytes)
RSA-2048	0.505	0.020	256	294
ECDSA-P256	0.023	0.057	71	91
Ed25519	0.033	0.089	64	32
ML-DSA-65 (FIPS 204)	0.671	0.162	3,309	1,952

Key establishment:

Algorithm	Full operation (ms)	Public key	Ciphertext
ECDH-P256	0.063	91	—
ECDH-P384	1.610	120	—
X25519	0.060	32	—
ML-KEM-768 (FIPS 203)	0.263	1,184	1,088

Certificates (really issued and DER-encoded during the run):

Algorithm	Certificate size	vs RSA-2048
RSA-2048	879 B	1.0×
ECDSA-P256	484 B	0.6×
ML-DSA-65	5,608 B	6.4×

The three findings that come out of this:

1. **Speed is not the blocker.** ML-DSA-65 signs at **1.3×** the cost of RSA-2048 — a rounding error in almost any service.
2. **Size is the blocker.** Its signature is **3,309 bytes against 256 — 12.9×** larger. A three-certificate TLS chain goes from **2,637 to 16,824 bytes**, which overflows TCP's ~14,600-byte initial congestion window and **adds a round trip to every fresh connection**. On a 60 ms link that is 60 ms added to every new user session. That is a real, budgetable consequence.
3. **Post-quantum is not universally slower.** ML-KEM-768 completes a full key establishment in 0.263 ms against 1.610 ms for ECDH-P384 — it is **6.1×** faster at a comparable security level.
It costs bytes, not cycles.

This is what "measured, not estimated" buys you. None of these three sentences could be written by a tool that quoted a datasheet.

Part 6 — The technical stack

6.1 What we actually use today

This matters: our prototype is **deliberately minimal**. That is a design choice we defend, not a shortcut we apologise for.

Layer	What we use	Why
Language	Python 3.12	Readable, batteries included, fastest for a beginner team to extend
Parsing	ast (Python standard library)	Real AST parsing with zero dependencies
Pattern matching	re (standard library)	56 compiled rules for cheap triage
Crypto + benchmark	cryptography (pyca) on OpenSSL 4.0.1	Ships NIST-standardised ML-KEM and ML-DSA <i>natively</i> — this is why we can measure PQC without exotic setup
Manifest	PyYAML	Human-writable service manifest
Output	JSON + CycloneDX 1.6	Machine-readable, standards-compliant
Dashboard	Plain HTML + CSS + JavaScript	No framework, no build step, no CDN, no fonts fetched, no telemetry
Deployment	Double-click <code>index.html</code>	That is the whole deployment

Total external dependencies: two. `cryptography` and `PyYAML`. Everything else is the Python standard library.

Why this is a strength, not a limitation: the tool runs with the network cable out. For NTRO — an intelligence organisation — the ability to scan classified source on an air-gapped machine with no telemetry and no cloud API is not a nice-to-have, it is the entry requirement. We demonstrate it rather than assert it.

6.2 What we would use at production scale

The team's original brief (`ECDAT_Team_Brief.md`) proposed a fuller architecture. That remains the right target for a real product; it is simply not what the prototype is. Be clear about which one you are describing.

```

Web UI (SvelteKit or React)
|
FastAPI controller + authentication
|
Lightweight local job queue
|

```

```

Scanner workers (Python; Go only if benchmarking proves it necessary)
├ Source/API rules + Tree-sitter AST    ← multi-language parsing
├ Dependency / config scanner
├ Certificate / key metadata parser
├ Container and limited binary scanner
└ Optional runtime / endpoint probes
    |
    SQLite metadata + DuckDB analytics + Parquet findings
    |
    Rule engine → CBOM → dashboard / report

```

Choice	Why it fits
Tree-sitter	Incremental multi-language parsing without writing a compiler per language — the route to Go, C/C++, JavaScript
SQLite	Zero-admin local database; portable, adequate for a small deployment
DuckDB / Parquet	Efficient local analytics over compressed findings
FastAPI	Simple API with automatic documentation
Docker Compose	One-command local deployment

Explicitly rejected for now: Kafka, OpenSearch, MongoDB, Kubernetes sandbox fleets.

Those solve

scale and availability problems we do not have. Adopting them early would be cargo-culting an enterprise stack.

6.3 On LLMs — read this carefully

There is no LLM anywhere in ECDAT's detection or scoring path. Not optional-off; absent.

The architecture *permits* an optional LLM layer for **explaining a verified finding to a non-specialist**, and the team brief sets the guardrails: the deterministic scanner creates the finding, only a small redacted snippet is ever sent, output is constrained to a JSON schema, every explanation must cite scanner evidence, and "unknown" is an acceptable answer.

The design test: *if the LLM is switched off, ECDAT must still discover, score, export and benchmark.* Today it always is, so it always does.

If a judge asks "is this just a ChatGPT wrapper?", the answer is short: **"No. There is no model in the detection path at all. Every finding comes from a compiled rule and an AST parse,**

and the rule

ID is printed next to the finding."

Part 7 — How to run it

7.1 First-time setup

You need **Python 3.12 or newer**.

```
pip install cryptography pyyaml
```

Important: the benchmark measures ML-KEM and ML-DSA using OpenSSL's *native* support.

That

requires a recent `cryptography` wheel built against **OpenSSL 3.5 or newer** (our demo machine has

OpenSSL 4.0.1). If your OpenSSL is older, the scan and dashboard still work perfectly — only the

post-quantum benchmark rows will be missing. Check with:

```
python -c "from cryptography.hazmat.backends.openssl.backend import backend;
print(backend.openssl_version_text())"
```

7.2 Running it

Option A — just look at the dashboard (nothing to install, nothing to run):

Double-click `proto/ui/index.html`. `data.js` is already committed, so the dashboard works immediately. **This is what you do on demo day.**

Option B — regenerate the dataset from scratch:

```
cd proto
python engine/run.py
```

Takes about 13 seconds. It scans every target, parses the certificates, runs the real benchmark, and

rewrites `ui/data.js`, `data/scan_results.json` and the CBOMs.

Option C — serve it over HTTP (identical result):

```
cd proto
python -m http.server 8765 --directory ui
```

Then open `http://localhost:8765`.

Option D — rebuild the single-file shareable version:

```
cd proto
python engine/build_artifact.py
```

Produces `ui/ecdat-console.artifact.html` — one self-contained file you can email.

7.3 Troubleshooting

Symptom	Cause	Fix
Dashboard says "data.js not found"	You opened <code>app.js</code> or the wrong file	Open <code>ui/index.html</code>
No post-quantum rows in the passport	OpenSSL too old	Upgrade <code>cryptography</code> ; the rest still works
<code>ModuleNotFoundError: yaml</code>	PyYAML not installed	<code>pip install pyyaml</code>
Changes to the UI do not appear	Browser cache	Hard reload (<code>Ctrl+Shift+R</code>)
The numbers changed after I re-ran	Timings and finding IDs are re-measured each run	Expected. Do not re-run on demo day.

Demo-day rule: do not run `engine/run.py` on the day of the presentation. The committed `data.js` contains the numbers our slides and script quote. Re-running produces new (equally valid but different) timings and would desynchronise the script.

Part 8 — How to build and extend it

This is the implementation guide. If you are picking up a task, start here.

8.1 Adding a new algorithm to the knowledge base

File: `engine/rules.py`, the `ALGORITHMS` dictionary.

```
"BLOWFISH": dict(
    family="symmetric",
    quantum="classically-broken",
    nist_pqc=None,
    note="64-bit block size; vulnerable to birthday attacks on long sessions
(Sweet32).",
    replacement="AES-256-GCM",
),
```

The four `quantum` values are the only ones allowed: `shor-broken`, `classically-broken`, `grover-reduced`, `quantum-safe`. Read the definitions at the top of `rules.py` before choosing.

8.2 Adding a new detection rule

File: `engine/rules.py`, the `RULES` list.

A rule needs: an `id` (which appears in the UI next to every finding it produces), the languages it applies to, a compiled regex, and what it means. Design principles:

1. **Prefer decoding over guessing.** If the call names its own algorithm — `getInstance("SHA256withRSA")` — capture the string and decode it with `resolve_jca()`. Do not hardcode an assumption.
2. **Capture group 1 is the payload.** The scanner reads `m.group(1)` when the rule has one.
3. **Give it a truthful ID.** `py-hash-sha1` tells a reviewer exactly what fired.
4. **Test on a real repository**, not only on a file you wrote to make it pass.

8.3 Adding a new language

This is the biggest single extension and the clearest roadmap item.

1. Add the file extensions to `LANG_BY_EXT` in `scanner.py`.
2. Add rules tagged with the new language.
3. Build a symbol index so findings can report the enclosing function. Python uses `ast`; Java uses a bounded regex look-around. For Go or JavaScript, **Tree-sitter** is the correct tool — it gives you a real parse tree for ~40 languages without writing a compiler.
4. Add the extension to the triage token list if the language's crypto APIs use different names.

Estimated effort: Go or JavaScript with Tree-sitter is roughly a two-week task for one person who

has done the tutorial first.

8.4 Writing a service manifest

File: `targets/<repo>/ecdat-manifest.yaml` . This is the artefact that makes the criticality layer

work, and it is written by hand by the asset owner.

```
organisation: Nivesh Financial Services
threat_horizon_years: 15          # Z
horizon_basis: "Board risk appetite, anchored to NIST IR 8547"

services:
  - name: payment-api
    description: "Settlement instruction signing and clearing-gateway
integration"
    path_patterns:
      - "services/payment-api/**"
    data_classification: financial      # public | internal | financial |
sensitive-pii
    business_function: payment-processing
    exposure: internet-facing          # offline | internal | internet-
facing
    data_lifetime_years: 7             # X -- PMLA transaction record
retention
    migration_time_years: 3            # Y -- HSM + counterparty
coordination
    migration_constraints:
      - "HSM firmware support for ML-DSA is unverified"
```

Every field maps directly onto a weight in Part 5.1. When two patterns match the same file, the **more specific** pattern wins.

8.5 Changing the scoring weights

File: `engine/criticality.py` , the three dictionaries at the top, and `run.py` for `QUANTUM_URGENCY` and `RISK_BUCKETS` .

Change a number, re-run `engine/run.py` , and the dashboard reflects it — including the weight tables,

which are rendered *from* those values. Nothing is hardcoded twice. If you change a weight, say so in

the demo: *"these live in `engine/criticality.py` ; an auditor can recompute every number by hand."*

8.6 Working on the dashboard

Files: `ui/app.js`, `ui/styles.css`, `ui/index.html`.

- No build step, no framework, no package manager. Edit and refresh.
- `app.js` is one IIFE. Each screen is a `viewX()` function returning an HTML string; `render()` paints it and `wire()` attaches the event handlers. If you add interactive elements, wire them in `wire()` — it is re-run after every render.
- **Always escape user-ish data with** `esc()` when building HTML strings.
- Colours come from CSS custom properties in `:root` and from the `RISK_COLOR` / `Q_COLOR` maps at the top of `app.js`. Keep those two in step — the same severity must never render in two colours.
- After changing the UI, rebuild the shareable copy: `python engine/build_artifact.py`.

8.7 Adding a step to the guided walkthrough

File: `ui/app.js`, the `DEMO` array.

```
{
  view: "cbom", repo: "nivesh-core", risk: "", openTop: false,
  title: "A catalogue other tools can read",
  say: "CycloneDX 1.6 added the cryptographic-asset type in 2024 for exactly this job..."
}
```

The rail renders the count automatically and the number keys extend to match. `openTop: true` makes

the step open the evidence drawer on the highest-risk finding in the selected target — it resolves by rank, not by a hardcoded ID, so a rescan will not break it.

8.8 The rules we hold ourselves to when writing code

- **Every number in the UI must be traceable to a line of code or a measurement.** If you cannot show where it came from, do not display it.
- `unknown` **is a valid, correct answer.** Never substitute a default to make a screen look fuller.
- **Never claim a finding is exploitable without proof.** We report *presence*, not exploitability.

- **Never display private key material or secrets**, even from our own demo files.
 - **Only scan repositories we are authorised to scan.** Our four targets are our own files and three public open-source projects.
 - **Keep the scan audit trail:** input hash, scanner version, ruleset version, timestamp.
-

Part 9 — Future plans

Phase 1 — Post-hackathon hardening (1–2 months)

Item	Why it matters
requirements.txt and a one-command installer	Right now a new teammate has to know to <code>pip install cryptography pyyaml</code>
A test suite with a labelled corpus	We currently cannot state precision or recall as a measured number. This is the single biggest credibility gap
Measure precision / recall / F1 per language	Target $\geq 90\%$ recall and $\geq 85\%$ precision on supported APIs
Remove <code>__pycache__</code> from version control	Tracked build artefacts churn on every run
CI check that <code>data.js</code> matches the committed engine output	Prevents the deck and the tool drifting apart

Phase 2 — Coverage (3–6 months)

- **More languages** via Tree-sitter: Go, JavaScript/TypeScript, C/C++.
- **Real parallelism.** The scanner is single-threaded today. Bounded worker concurrency is the honest next step for large monorepositories.
- **Container and binary scanning** — start with file type, strings, linked libraries and embedded certificates, labelled explicitly as lower confidence.
- **Scale validation** — a properly measured run on a 10,000–50,000 file repository. Today this is an aspiration, and we say so.

Phase 3 — The two flagship features (6–12 months)

Sandboxed migration rehearsal. Today the passport measures algorithms directly. The real version

runs a live TLS handshake against an `oqs-provider` endpoint in an isolated Docker network marked

`internal: true` — making "isolated test environment" an *inspectable property of the architecture*

rather than a sentence in a pitch. If a client library cannot negotiate the hybrid group at all, **that failure is itself the finding** — precisely the "blocker: legacy TLS/runtime" result type.

CI/CD crypto-agility gate. A pull request that reintroduces MD5, SHA-1 or RSA-1024 fails the build, with the CBOM diff posted as a comment. This turns ECDAT from an audit tool into a preventative control, which is where the real value is.

Phase 4 — Enterprise and adoption (12+ months)

- **CMDB / asset-registry integration**, replacing the hand-written manifest while keeping manifest tagging as the accessible default for organisations without a CMDB.
- **Role-based access control and a real audit trail** for government deployment.
- **Compliance mapping** — exportable reports mapped to CERT-In, RBI and SEBI requirements.
- **Optional connectors** for network, HSM and cloud discovery — opt-in, clearly logged, keeping the sovereign-by-design posture intact.
- **Adoption model** — open-source the core scanner, offer a supported variant. This is the model CBOMkit and the PQCA have already validated as workable.

The three unconventional ideas worth considering

A · Adversarial mode. Let a judge or a customer drop *their own* file into the console and watch it scan live, air-gapped. Our strongest claim is "we scanned real code, not our code" — letting someone supply the input turns that from an assertion into a demonstration. It also puts the honest degradation story on stage: their file has no manifest, so it comes back `unknown`, exactly as designed.

B · The regret curve. Instead of Z producing a verdict, plot *cost of acting now* against *cost of being late* per service and mark where the curves cross. Everyone argues about when the quantum computer arrives; this reframes it as *"at what Z does starting late become more expensive than*

starting early?" — the question a CFO actually signs against. It turns our biggest honest weakness
(nobody knows Z) into our sharpest output.

C · Signed evidence bundle. One command emits the CBOM, passport, manifest, rule catalogue and a hash of every scanned file into a single Merkle-rooted archive. An auditor can then prove *this finding came from this byte of this file at this commit*, months later. For NTRO this is the difference between a scanner and an artefact admissible in a compliance review — and it is cheap, because the incremental scanner already computes those hashes.

Part 10 — Splitting the work across six people

10.1 The principle

Six people cannot all edit `scanner.py`. The split below gives each person **one file or screen they own, one concept they are the team expert on, and one part of the pitch they deliver**. Ownership means: you make the changes, you review anyone else's changes there, and you answer the jury's questions about it.

10.2 The six roles

Role 1 — Scanner & Rules Lead

Owns: `engine/scanner.py`, `engine/rules.py`

Expert on: static analysis, ASTs, how a finding is produced

Builds: new detection rules, new algorithm profiles, the triage/regex/AST pipeline, new language support

Must be able to explain: *"Why is this not just a grep?"* — the four stages, and why AST resolution means we can report `key_size=2048` rather than guessing.

Learning path: Python `ast` module tutorial → the "Green Tree Snakes" AST guide → read `scan_source_file()` line by line → add one rule and watch it fire.

Role 2 — Cryptography & Risk Lead

Owns: `engine/criticality.py`, the scoring sections of `engine/run.py`

Expert on: PQC, Shor vs Grover, Mosca's inequality, the criticality and risk formulas

Builds: the weight tables, the manifest matcher, the Mosca calculator, the risk score

Must be able to explain: every arithmetic step in Part 5, from memory, at a whiteboard.

Learning path: NIST PQC project pages → FIPS 203/204 abstracts (just the abstracts) → NIST IR

8547 → Mosca's 2018 IEEE paper → then read `criticality.py`, which is only 200 lines.

Role 3 — Artefacts & CBOM Lead

Owns: `engine/artefacts.py`, `engine/cbom.py`

Expert on: X.509 certificates, TLS configuration, CycloneDX, dependency manifests

Builds: certificate parsing, Key Usage checks, expiry detection, the CBOM export

Must be able to explain: what a certificate contains, why RFC 5280 Key Usage flags matter, and

why emitting a *standard* format is a strategic decision rather than a formatting choice.

Learning path: run `openssl x509 -in cert.pem -text -noout` on our four demo certs → read RFC

5280 section 4.2.1.3 → read the CycloneDX 1.6 cryptographic-asset spec page.

Role 4 — Benchmark & Migration Passport Lead

Owns: `engine/bench.py`, the Migration Passport screen's content

Expert on: measurement methodology, ML-KEM/ML-DSA performance and size, the TLS size problem

Builds: the benchmark harness, certificate issuance under each algorithm, the passport record

Must be able to explain: the measurement method (warm up 5×, sample until both 30 iterations and

0.35 s have elapsed, report the median, retain mean/p95/stddev), and the congestion-window argument.

Learning path: read `bench.py` top to bottom — it is the most self-contained module → run it alone → change the sample count and watch the medians stay stable. That stability *is* the methodology argument.

Role 5 — Dashboard & UX Lead

Owns: `ui/app.js`, `ui/styles.css`, `ui/index.html`, the guided walkthrough

Expert on: how the story is told on screen; the evidence drawer

Builds: the seven screens, the drawer, the walkthrough, the design system

Must be able to explain: why there is no framework and no build step, and why that is a deliberate

air-gap decision rather than laziness.

Learning path: open `app.js`, find `viewOverview()`, change one string, refresh. Repeat until

the
render → wire cycle is obvious. Then read `openFinding()` — the drawer is the product.

Role 6 — Demo, Documentation & QA Lead

Owns: `README.md` , this guide, the pitch plan, the walkthrough script, the deck
Expert on: what we claim and what we refuse to claim
Builds: the demo run-book, the Q&A bank, the rehearsal schedule, the claim-discipline checklist;
runs the pre-demo verification pass before every rehearsal and on the day
Must be able to explain: every limitation on the Method & Limits screen, cheerfully and first.
Learning path: read Part 11 of this guide, then watch each of the other five demo their own area
and write down every claim they make. Anything that cannot be traced to code or a measurement gets challenged.

10.3 Where the roles connect

These are the seams where two people must agree. Get these wrong and work gets done twice.

Interface	Between	The contract
The finding record	Role 1 → Roles 2, 5	Scanner emits <code>algorithm</code> , <code>function</code> , <code>file</code> , <code>line</code> , <code>col</code> , <code>snippet</code> , <code>match</code> , <code>symbol</code> , <code>rule_id</code> , <code>language</code> . Roles 2 and 5 consume it. Any field added must be added here first.
The algorithm table	Role 1 ↔ Role 2	<code>ALGORITHMS</code> in <code>rules.py</code> is the single source of truth for quantum status. Role 2 must not maintain a second copy.
The criticality record	Role 2 → Role 5	Emits <code>score</code> , <code>bucket</code> , <code>confidence</code> , <code>evidence</code> , <code>working{}</code> , <code>service</code> , <code>owner</code> . The <code>working</code> dict is what the drawer renders as the arithmetic.
The benchmark record	Role 4 → Role 5	Signatures are keyed on <code>name</code> ; certificates are keyed on <code>algorithm</code> . (These differ — the UI has separate lookups for exactly this reason.)
<code>data.js</code>	Roles 1–4 → Role 5	Generated only. Never hand-edited. If the UI needs a new number, it is added in <code>run.py</code> .
The claim list	Role 6 → everyone	Nobody puts a claim on a slide or says it out loud unless Role 6 can trace it to code or a measurement.

10.4 A workable weekly rhythm

Day	Activity	Who
Monday	30-min standup: what each person shipped, what is blocked	All six
Tue–Thu	Build in your own area; make small changes rather than large ones	Individual
Friday	Integration run. Role 6 runs <code>engine/run.py</code> end to end, opens the dashboard, walks all seven steps, and reports anything broken	All six watch
Friday	Claim review. Any new sentence on a slide gets traced to code, or removed	Role 6 leads

10.5 If you are completely new

Everyone should do this first, before touching anything:

1. Run `python engine/run.py`. Watch it finish in 13 seconds.
2. Open `ui/index.html`. Press **Guided walkthrough**. Walk all seven steps with the arrow keys.
3. Open the Crypto Inventory, click any row, and read the whole drawer — top to bottom.
4. Open `engine/criticality.py` and find the number 5 next to `sensitive-pii`.
5. Find that same 5 in the drawer you just read.

When you can trace one number from the source file to the screen, you understand this product. That takes about forty minutes and it is the best-spent time on this project.

Part 11 — The honesty rules

This section exists because **the fastest way to lose a technical jury is one overstated claim.**

A

judge who catches you exaggerating once will re-examine everything else you said.

11.1 Claims we make, and what backs each one

Claim	Evidence
"We scanned 3,111 real source files"	paramiko, JWT, Django cloned unmodified from upstream; the telemetry is on the Command Centre screen

Claim	Evidence
"Every finding carries file, line, column and the matched text"	Open any drawer
"17 X.509 certificates parsed with real key sizes and Key Usage flags"	engine/artefacts.py , and the certificates are on disk
"Every benchmark number was measured on this machine"	engine/bench.py , and the environment is printed on the passport screen
"8× incremental speedup"	Measured back to back in the same run: 3.77 s vs 0.47 s
"We emit standard CycloneDX 1.6"	Download the CBOM and read it
"Criticality is manifest-backed with honest confidence labelling"	Switch to paramiko and watch it degrade to 0/17/38
"Mosca's inequality is interactive, not buried"	Drag the Z slider

11.2 Claims we do NOT make — ever

Never say	Say instead
"It automatically generates the fix" / "codemod" / "patch code"	"It produces a developer-reviewed migration blueprint and measures what the change costs"
"We support C++" (or Go, or JavaScript, or binaries)	"Python and Java, plus TLS config and X.509. Everything else is roadmap, and the tool says so on its own Limits screen"
"Sandboxed migration rehearsal"	"Measured migration cost, benchmarked on the host. A sandboxed handshake is the next step, not a claim we make today"
"Parallelised worker threads"	"Content-hash incremental rescanning, measured at 8×. Parallelism is planned, not built"
"PQC migration is legally mandated in India by 2030"	"NSA CNSA 2.0's 2033 deadline is the firmest published one, and it binds US national-security systems. Everywhere else it is growing pressure, not a mandate"
"This finding is exploitable"	"This algorithm is present at this line. Exploitability requires proof we have not gathered"

Never say	Say instead
"Nivesh is a real financial estate"	"Nivesh is fictional. Its files are real and really scanned; we wrote the organisation and its manifest for this demonstration"
"We know when quantum computers will break RSA"	"Nobody does. That is why Z is a slider with its basis printed on screen"
"The first tool to do migration testing"	"A focused, accessible and measurable version of it"

11.3 The six limitations we state before anyone asks

These are on the Method & Limits screen, in the tool, in our own words:

1. **Two languages.** Python and Java, plus TLS config and X.509. Not Go, C/C++, JavaScript or binaries.
2. **Static analysis has a ceiling.** Runtime-constructed algorithm names and reflection are invisible to this approach.
3. **Nivesh Financial Services is fictional.** Its files are real and really scanned; the organisation and its manifest were written for this demonstration.
4. **The manifest is filled in by hand.** CMDB integration does not exist yet.
5. **No live TLS handshake.** The benchmark measures algorithms directly.
6. **One machine, one stack.** An HSM or a constrained device will produce different numbers — which is why the passport records its own environment.

Say these first, cheerfully. A team that names its own limits reads as competent. A team that gets them extracted by a judge reads as evasive.

Part 12 — Quick reference card

Print this. Keep it with you on demo day.

Identity: SIH26164 · ECDAT · NTRO · Blockchain & Cybersecurity · Software · Team Asterisks

The pitch: finds cryptography in source code, works out what it protects, decides how urgently it must be replaced, and measures what replacing it would cost.

The numbers:

4 repos · 3,111 files · 206 findings · 120 quantum-vulnerable · 17 certs · 56 rules · 37 algorithms
3.77 s clean scan · 0.47 s incremental (8×) · ~13 s total pipeline

The formulas:

criticality = (data + function) × exposure
risk = quantum_urgency × (criticality / 20) × 2 + mosca_pressure
Mosca: $X + Y > Z \rightarrow$ already too late

The three killer facts:

1. MD5 and SHA-1 are broken TODAY — they outrank a threat needing hardware nobody has built
2. treasury-archive is only "high" criticality but 14 years past the Mosca line — severity ≠ urgency
3. ML-DSA-65: 1.3× slower to sign, 12.9× larger signature — **size is the blocker, not speed**

The four quantum statuses:

classically-broken (urgency 5) · shor-broken (4) · grover-reduced (2) · quantum-safe (0)

The three confidence labels:

manifest-confirmed · path-heuristic · unknown

The three NIST standards:

FIPS 203 = ML-KEM (key exchange) · FIPS 204 = ML-DSA (signatures) · FIPS 205 = SLH-DSA (backup)

The walkthrough keys: → next · ← back · 1-7 jump · Esc exit

If you do not know an answer: *"I don't know — that is outside what we measured. What I can tell you is what we did measure, which is..."* That answer never loses points. Guessing does.